CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

Practical Aim:

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

Problem 1 :

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks.

Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practicle-3

Output:

```
Enter size of array:
6
Enter array element:
4
3
6
7
2
9
Insertion Sort: 2 3 4 6 7 9
Bubble Sort: 2 3 4 6 7 9
Selection Sort: 2 3 4 6 7 9
PS C:\Users\HP\OneDrive\Documents\DSA\Practicle-3>
```

Problem Faced:

Understanding the differences between Bubble Sort, Selection Sort, and Insertion Sort and comparing their performance was challenging. It was also necessary to determine which method works best when the data is already sorted.

Outcome/Conclusion:

The outcome was a better understanding of sorting algorithms and their efficiency. All three methods correctly sorted the marks, and Bubble Sort was found to be the fastest for an already sorted list because it can stop early when no swaps are required.

Problem 2 :

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practicle-3

Output:

```
Enter a array Size:
5
Enter array element:
0
1
1
1
0
Sorted Colors: 0 0 1 1 1
PS C:\Users\HP\OneDrive\Documents\DSA\Practicle-3>
```

Problem Faced:

The main challenge in this task was understanding how to rearrange the colour codes without using any extra storage. It was also necessary to analyze the difference between a two-pass counting approach and a one-pass solution and determine how the elements could be moved efficiently within the same array.

Outcome/Conclusion:

The outcome was a clear understanding of in-place sorting techniques for arrays containing only 0s, 1s, and 2s. The colour codes were successfully arranged in the required order, and it was learned that the Dutch National Flag algorithm can sort the array in a single pass while using no extra storage.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

Sol: Bubble Sort. If no swaps occur in a pass, the list is already sorted and the algorithm stops. Selection Sort and Insertion Sort still need to continue their comparisons, so they cannot use this early-stop check as effectively.

[Problem 2] If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

Sol: The first pass counts the number of 0s, 1s, and 2s, and the second pass rearranges them based on those counts.

And Yes. The Dutch National Flag algorithm uses three pointers to place 0s, 1s, and 2s in their correct positions in a single pass without extra storage.